

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing



```
hemasai@Hemasai: ~/OSSP_1 × + ▾
GNU nano 8.7.1 src/skill1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t child_pid;
    int status;

    printf("=====\n");
    printf("        OSSP - SKILL 1\n");
    printf("        LINUX COMMAND EXECUTION\n");
    printf("=====\n\n");

    printf("[PARENT PROCESS]\n");
    printf("Parent PID: %d\n", getpid());

    child_pid = fork();

    if (child_pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (child_pid == 0)
    {
        printf("\n[CHILD PROCESS]\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        printf("Executing command: ls\n\n");
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```

        perror("exec");
        exit(EXIT_FAILURE);
    }
    else
    {
        printf("Child PID: %d\n", child_pid);
        printf("Status : Waiting for child...\n");

        if (waitpid(child_pid, &status, 0) == -1)
        {
            perror("waitpid");
            return 1;
        }

        printf("\n[PROCESS STATUS]\n");

        if (WIFEXITED(status))
        {
            printf("Child process completed.\n");
            printf("Exit status: %d\n", WEXITSTATUS(status));
        }
        else
        {
            printf("Child process did not exit normally.\n");
        }
    }

    printf("\n=====\n");
    printf("        SKILL 1 EXECUTION COMPLETED\n");
    printf("=====\\n");

    return 0;
}

```

```

hemasai@Hemasai:~/OSSP_Assignment/OSSP_Skill_1$ pwd
/home/hemasai/OSSP_Assignment/OSSP_Skill_1
hemasai@Hemasai:~/OSSP_Assignment/OSSP_Skill_1$ ls src
skill1.c
hemasai@Hemasai:~/OSSP_Assignment/OSSP_Skill_1$ gcc src/skill1.c -o skill1
hemasai@Hemasai:~/OSSP_Assignment/OSSP_Skill_1$ ls
docs  results  skill1  src
hemasai@Hemasai:~/OSSP_Assignment/OSSP_Skill_1$ ./skill1
=====
                OSSP - SKILL 1
                LINUX COMMAND EXECUTION
=====

[PARENT PROCESS]
Parent PID: 1950
Child PID: 1951
Status : Waiting for child...

[CHILD PROCESS]
Child PID: 1951
Parent PID: 1950
Executing command: ls

docs  results  skill1  src

[PROCESS STATUS]
Child process completed.
Exit status: 0

=====
                SKILL 1 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment/OSSP_Skill_1$ |

```

2. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```
hemasai@Hemasai: ~/OSSP_1 × + v
GNU nano 8.7.1 skill2.c
#include <stdio.h>
#include <string.h>

#define INPUT_SIZE 256

int main()
{
    char input[INPUT_SIZE];

    printf("=====\n");
    printf("        OSSP - SKILL 2\n");
    printf("        INTERACTIVE SHELL LOOP\n");
    printf("=====\n");

    printf("Shell started successfully.\n");
    printf("Type 'exit' to terminate the shell.\n\n");

    while (1)
    {
        printf("OSSP-Shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            printf("\nEnd of input detected.\n");
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("\n[EXIT COMMAND]\n");
            printf("Exit command received.\n");
        }
    }
}

[ Wrote 57 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Lo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go
```

```
        printf("\nEnd of input detected.\n");
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
    {
        printf("\n[EXIT COMMAND]\n");
        printf("Exit command received.\n");
        break;
    }

    if (strlen(input) == 0)
    {
        printf("Empty command ignored.\n\n");
        continue;
    }

    printf("\n[COMMAND RECEIVED]\n");
    printf("Command: %s\n", input);
    printf("Command length: %zu characters\n", strlen(input));
    printf("Status: Input accepted successfully.\n\n");
}

printf("Shell loop terminated.\n");

printf("\n===== \n");
printf("        SKILL 2 EXECUTION COMPLETED\n");
printf("===== \n");

return 0;
}
```

```

hemasai@Hemasai:~/OSSP_Assignment$ ./skill2
=====
                OSSP - SKILL 2
            INTERACTIVE SHELL LOOP
=====
Shell started successfully.
Type 'exit' to terminate the shell.

OSSP-Shell> ls

[COMMAND RECEIVED]
Command: ls
Command length: 2 characters
Status: Input accepted successfully.

OSSP-Shell> pwd

[COMMAND RECEIVED]
Command: pwd
Command length: 3 characters
Status: Input accepted successfully.

OSSP-Shell> hello world

[COMMAND RECEIVED]
Command: hello world
Command length: 11 characters
Status: Input accepted successfully.

OSSP-Shell> exit

[EXIT COMMAND]
Exit command received.
Shell loop terminated.

=====
                SKILL 2 EXECUTION COMPLETED
=====

```

-
3. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

```
=====
                O SSP - SKILL 3
            COMMAND HISTORY & MEMORY
=====
```

History system initialized.
Dynamic buffer capacity: 4

Enter commands to store in history.
Type 'done' when finished.

O SSP-History> ls
[HISTORY STORED]
Command: ls
History entries: 1

O SSP-History> pwd
[HISTORY STORED]
Command: pwd
History entries: 2

O SSP-History> date
[HISTORY STORED]
Command: date
History entries: 3

O SSP-History> whoami
[HISTORY STORED]
Command: whoami
History entries: 4

```
=====
```

[COMMAND RECALL]
Enter history number to recall: 2

[PREVIOUS COMMAND]
Recalled command: pwd

[INPUT BUFFER UPDATE]
Input buffer updated with: pwd

[NEXT COMMAND TEST]
Next command: date
Input buffer moved to next command.

[MEMORY CLEANUP]
All history nodes released.
All dynamically allocated command buffers released.
Memory cleanup completed successfully.

```
=====
```

SKILL 3 EXECUTION COMPLETED

```
=====
```

```
hemasai@Hemasai: ~/OSSP_1 X + v
History entries: 1

OSSP-History> pwd
[HISTORY STORED]
Command: pwd
History entries: 2

OSSP-History> date
[HISTORY STORED]
Command: date
History entries: 3

OSSP-History> whoami
[HISTORY STORED]
Command: whoami
History entries: 4

OSSP-History> uname
[HISTORY BUFFER]
Buffer resized to capacity: 8
[HISTORY STORED]
Command: uname
History entries: 5

OSSP-History> done

=====
COMMAND HISTORY
=====
1. ls
2. pwd
3. date
4. whoami
5. uname
=====

[COMMAND RECALL]
Enter history number to recall: |
```

-
4. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```
=====
                OSSP - SKILL 4
            TOKENIZER & PARSER DEMO
=====
```

Enter a command to parse.

Example: `ls -l src`

OSSP-Parser> `ls -l src`

[INPUT]

Original command: `ls -l src`

[TOKENIZATION]

Identifying whitespace delimiters...

Tokenization completed successfully.

[TOKEN STREAM]

Token 1: [`ls`]

Token 2: [`-l`]

Token 3: [`src`]

Total tokens: 3

[VALIDATION]

Token stream is valid.

[PARSER]

Building parser structure...

Syntax validation successful.

Parser structure generated successfully.

[EXECUTION STRUCTURE]

Command: `ls`

Arguments:

```

[INPUT]
Original command: ls -l src

[TOKENIZATION]
Identifying whitespace delimiters...
Tokenization completed successfully.

[TOKEN STREAM]
Token 1: [ls]
Token 2: [-l]
Token 3: [src]
Total tokens: 3

[VALIDATION]
Token stream is valid.

[PARSER]
Building parser structure...
Syntax validation successful.
Parser structure generated successfully.

[EXECUTION STRUCTURE]
Command: ls
Arguments:
  Argument 1: -l
  Argument 2: src

[DEBUG PARSING OUTPUT]
Input accepted.
Tokens validated.
Command identified.
Arguments identified.
Execution structure ready.

=====
                SKILL 4 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$ |

```

-
5. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```

=====
                OSCP - SKILL 5
            QUOTED STRING PARSER
=====

Supported features:
- Single quotes
- Double quotes
- Preserved spaces
- Variable expansion rules
- Nested token parsing

Examples:
'Hello World'
"Hello World"
'Literal $HOME'
"Value $HOME"

OSSP-Quote> 'Hello World'

[ORIGINAL INPUT]
'Hello World'

[QUOTED STRING PARSING]
Quoted parsing completed successfully.

[PARSED TOKENS]
Token 1: [Hello World]  Type: SINGLE QUOTED

[VARIABLE EXPANSION]

[PARSING VALIDATION]
Token 1: literal content preserved.

[EDGE CASE VALIDATION]
Quote boundaries validated.
Spaces inside quotes preserved.
Variable expansion rules checked.
Quoted command structure is valid.

=====
                SKILL 5 EXECUTION COMPLETED
=====

```

-
6. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```

hemasai@hemasai:~/OSSP_Assignments_7/skill6
=====
OSSP - SKILL 6
ESCAPE & PROCESS EXECUTION
=====

[PART 1 - ESCAPE PROCESSING]
Enter text with escape characters.
Example: hello\ world

OSSP-Escape> hello\ world

[ESCAPE RESULT]
Original : hello\ world
Processed: hello world

[PRESERVED CHARACTERS]
Character 1: h
Character 2: e
Character 3: l
Character 4: l
Character 5: o
Character 6:
Character 7: w
Character 8: o
Character 9: r
Character 10: l
Character 11: d

Escape processing completed successfully.
=====

[PART 2 - PROCESS EXECUTION]
=====
Enter a command with arguments.
Example: ls -l

OSSP-Execute> ls -l

[COMMAND PARSING]
Argument count: 2
Argument 1: ls
Argument 2: -l

[PROCESS CREATION]

[PARENT PROCESS]
Parent PID: 2253
Child PID: 2254
Waiting for child process...

[CHILD PROCESS]
Child PID: 2254
Parent PID: 2253
Executing command: ls
total 136
drwxr-xr-x 5 hemasai hemasai 4096 Oct  6 03:24 OSSP_Skill_1
-rwxr-xr-x 1 hemasai hemasai 16344 Oct  6 03:29 skill2
-rw-r--r-- 1 hemasai hemasai 1422 Oct  6 03:28 skill2.c
-rwxr-xr-x 1 hemasai hemasai 16800 Oct  6 03:34 skill3
-rw-r--r-- 1 hemasai hemasai 4955 Oct  6 03:33 skill3.c
-rwxr-xr-x 1 hemasai hemasai 16600 Oct  6 03:38 skill4
-rw-r--r-- 1 hemasai hemasai 5238 Oct  6 03:37 skill4.c
-rwxr-xr-x 1 hemasai hemasai 16500 Oct  6 03:40 skill5
-rw-r--r-- 1 hemasai hemasai 6105 Oct  6 03:40 skill5.c
-rwxr-xr-x 1 hemasai hemasai 16600 Oct  6 03:44 skill6
-rw-r--r-- 1 hemasai hemasai 4348 Oct  6 03:43 skill6.c

[PROCESS STATUS]
Child process completed.
Exit status: 0

=====
SKILL 6 EXECUTION COMPLETED
=====

```

7. Process Synchronization and Child Process Monitoring Using waitpid().

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

[illegible]

8. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```

OSSP - SKILL 8
VARIABLE & BUILT-IN DISPATCH
=====

[VARIABLE SYSTEM]
Variables initialized: 3

Available variables:
HOME=/home/hemasai
USER=hemasai
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/lat/lat:/mnt/c/Program Files/WindowsApps/MicrosoftCorporationII/WindowsSubsystemForLinux_2.7.8.0_x64_8weh
.../System32/OpenSSH/ /mnt/c/Program Files/dotnet/ /mnt/c/Program Files/Git/cmd/ /mnt/c/Program Files/GitHub CLI/ /mnt/c/Users/Hemas/AppData/Local/nvm/ /mnt/c/nwmm/nodesjs/ /mnt/c/Program Files/nodejs/ /mnt/c/Program
.../Files/MySQL/Shell.8.0/bin/ /mnt/c/Users/Hemas/AppData/Local/Programs/Python/Launcher/ /mnt/c/Users/Hemas/AppData/Local/Programs/Python/Python32/Scripts/ /mnt/c/Users/Hemas/AppData/Local/Programs/Pyth
.../onPython32/ /mnt/c/Users/Hemas/AppData/Local/Herms/hermes-agent/venv/Scripts/ /mnt/c/Users/Hemas/AppData/Local/Herms/bin/ /mnt/c/Users/Hemas/AppData/Local/Microsoft/
.../WindowsCommon-HostBinaries/x64/ /mnt/c/Users/Hemas/AppData/Local/Programs/Microsoft VS Code/bin/ /mnt/c/Users/Hemas/AppData/Local/Programs/Antigravity IDE/bin/ /mnt/c/Users/Hemas/AppData/Local/GitHubDesktop/bin/ /mnt/c/Users/Hemas/
.../AppData/Local/Microsoft/WinGet/Packages/Gyan.Ffmpeg_Microsoft.WinGet.Source_Bwekyb3d8bbwe/Ffmpeg-8.1.1-full_build/bin/ /mnt/c/Users/Hemas/AppData/Local/nvm/ /mnt/c/nwmm/nodesjs/ /mnt/c/Users/Hemas/AppData/Local/
.../Programs/GitLlama/ /mnt/c/Users/Hemas/AppData/Local/PowerToys/OSModules/ /mnt/c/Users/Hemas/AppData/Local/Microsoft/WinGet/Packages/oschartz18612.Poppler_Microsoft.WinGet.Source_8wekyb3d8bbwe/poppler-25.07.0/L
.../library/bin/ /mnt/c/Users/Hemas/AppData/Roaming/npm/ /snap/bin

[BUILT-IN DISPATCH TABLE]
=====

Command      Handler

pwd           builtin_pwd()
cd            builtin_cd()
export        builtin_export()
exit          builtin_exit

=====

[VARIABLE EXPANSION TEST]
=====

Enter text containing a variable.
Example: My home is $HOME

OSSP-Variable-> My home is $HOME

[ORIGINAL TOKEN]
My home is $HOME
(VARIABLE FOUND)
$HOME = /home/hemasai

[UPDATED TOKEN]
My home is /home/hemasai

[EXPANSION VALIDATION]
Variable expansion completed.

```

```
[EXPANSION VALIDATION]
Variable expansion completed.

=====
[BUILTIN COMMAND DISPATCH]
=====
Enter a built-in command.
Examples: pwd, cd, export, exit

OSSP-Builtin> ./skill8

[INVALID BUILTIN]
Command './skill8' is not a supported built-in.

=====
                SKILL 8 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$ |
```

-
9. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```

henasai@henasai:~/OSSP_Assignment$ ./skill9
=====
O SSP - SKILL 9
DIRECTORY & BUILTIN COMMANDS
=====
[INITIAL WORKING DIRECTORY]
/home/henasai/OSSP_Assignment
=====
[BUILTIN DISPATCH TABLE]
=====
Command      Handler
=====
pwd          showCurrentDirectory()
cd           changeDirectory()
cd -         changeDirectory(previous)
exit         terminate program
=====
[DIRECTORY NAVIGATION TEST]
=====
Available tests:
pwd
cd <directory>
cd -
exit
O SSP-Skill9> pwd
[DISPATCH]
Command identified as built-in: pwd
Executing pwd in-process.
[BUILTIN: pwd]
Current directory: /home/henasai/OSSP_Assignment
O SSP-Skill9> cd /tmp
[DISPATCH]
Command identified as built-in: cd
Executing cd in-process.
[BUILTIN: cd]
Directory changed successfully.
[BUILTIN: pwd]
Current directory: /tmp
O SSP-Skill9> cd -
[DISPATCH]
Command identified as built-in: cd
Executing cd in-process.
[BUILTIN: cd]
Directory changed successfully.
[BUILTIN: pwd]
Current directory: /home/henasai/OSSP_Assignment
O SSP-Skill9> pwd
[DISPATCH]
Command identified as built-in: pwd
Executing pwd in-process.
[BUILTIN: pwd]
Current directory: /home/henasai/OSSP_Assignment
O SSP-Skill9> cd /abc/xyz/notexist
[DISPATCH]
Command identified as built-in: cd
Executing cd in-process.
cd: No such file or directory
O SSP-Skill9> hello
[INVALID COMMAND]
Command 'hello' is not a supported built-in.
Available built-ins: pwd, cd, exit
O SSP-Skill9> exit
[DISPATCH]
Command identified as built-in: exit
Executing exit in-process.
Shell state terminated.
=====
SKILL 9 EXECUTION COMPLETED
=====
henasai@henasai:~/OSSP_Assignment$ |

```

10. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```

=====
OSSP - SKILL 10
DIRECTORY, EXIT & EXPORT
=====

[PART 1 - CURRENT DIRECTORY]

[CURRENT DIRECTORY]
Path: /home/hemasai/OSSP_Assignment

[STATE]
Current working directory retrieved successfully.
Shell state is available for further operations.

=====

[PART 2 - EXPORT COMMAND]
=====

Enter an export command.
Example: export MYVAR=HelloWorld

OSSP-Export> export MYVAR=HelloWorld

[EXPORT PARSING]
Input: export MYVAR=HelloWorld
Variable name: MYVAR
Variable value: HelloWorld

[NEW VARIABLE]
Variable does not currently exist.

[ENVIRONMENT UPDATED]
Export successful.

[VERIFICATION]
MYVAR=HelloWorld

[CHILD PROCESS TEST]
Child PID: 2511
Child received exported variable:
MYVAR=HelloWorld
Child completed successfully.
Child exit status: 0

=====

[CLEANUP]
=====
Temporary buffers cleared.
Resources cleaned up successfully.

[EXIT REQUEST]
Exit request processed successfully.
Program state verified.

=====

SKILL 10 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$ |

```

11. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```
=====
                        OSCP - SKILL 11
                        HISTORY & PIPELINE MANAGEMENT
=====

[PART 1 - COMMAND HISTORY]

Enter commands for history.
Type 'done' when finished.

OSSP-History> ls
[HISTORY STORED]
Command: ls
History count: 1

OSSP-History> pwd
[HISTORY STORED]
Command: pwd
History count: 2

OSSP-History> whoami
[HISTORY STORED]
Command: whoami
History count: 3

OSSP-History> done

=====
[COMMAND HISTORY]
=====
1  ls
2  pwd
3  whoami

History count: 3
History capacity: 3

[HISTORY RETRIEVAL]
Requested entry: 2
Retrieved command: pwd

[HISTORY VALIDATION]
History structure is consistent.
All stored entries are valid.
```

-
12. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

```

=====
SKILL 11 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$ nano skill12.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc -Wall -Wextra skill12.c -o skill12
hemasai@Hemasai:~/OSSP_Assignment$ ./skill12
=====
OSSP - SKILL 12
PIPE & STREAM MANAGEMENT
=====

[PIPELINE SUPPORT]
Create pipes
Redirect streams
Execute connected commands
Close descriptors
Synchronize processes
Support multiple pipes

Enter a pipeline command.
Example 1: ls | wc -l
Example 2: ls | grep .c | wc -l

OSSP-Skill12> ls | wc -l

=====
[PIPELINE STRUCTURE]
=====
Number of commands: 1

Stage 1: ls

[PIPELINE VALIDATION]
Stage 1 validated: ls
Pipeline configuration is valid.

```

```

=====
OSSP - SKILL 12
PIPE & STREAM MANAGEMENT
=====

[PIPELINE SUPPORT]
Create pipes
Redirect streams
Execute connected commands
Close descriptors
Synchronize processes
Support multiple pipes

Enter a pipeline command.
Example 1: ls | wc -l
Example 2: ls | grep .c | wc -l

OSSP-Skill12> ls | wc -l

=====
[PIPELINE STRUCTURE]
=====
Number of commands: 1

Stage 1: ls

[PIPELINE VALIDATION]
Stage 1 validated: ls
Pipeline configuration is valid.

=====
[PIPELINE EXECUTION]
=====

[PIPE CREATION]

[PROCESS CREATION]
Process 1 created. PID = 2598

[PARENT DESCRIPTOR CLEANUP]

[PROCESS SYNCHRONIZATION]
OSSP_Skill1 skill10.c skill11.c skill12.c skill12.c skill13.c skill14.c skill15.c skill16.c skill17.c skill18.c skill19.c
skill10 skill11 skill12 skill13 skill14 skill15 skill16 skill17 skill18 skill19
Process 1 completed. Exit status: 0
=====

```

13. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

```

hemasai@hemasai:~/OSSP_Assignment$ ./skill13
=====
                OSSP - SKILL 13
            INPUT & OUTPUT REDIRECTION
=====

[PART 1 - INPUT REDIRECTION]

Create a test input file first.
Enter a command using < redirection.
Example: wc -l < input.txt

OSSP-Input> wc -l < input.txt

[INPUT REDIRECTION PARSING]
Command : wc -l
Input file: input.txt

[INPUT FILE]
File opened successfully.
File descriptor: 3

[STDIN REDIRECTION]
Standard input redirected successfully.
[CHILD PROCESS]
Executing: wc
1

[STDIN RESTORED]
Original standard input restored.
Child exit status: 0

=====
[PART 2 - OUTPUT REDIRECTION]
=====

Enter a command using > redirection.
Example: ls > output.txt

OSSP-Output> 1

[ERROR]
Output redirection '>' was not found.
hemasai@Hemasai:~/OSSP_Assignment$ |

```

14. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

```

hemasai@hemasai: ~/OSSP_Assignment$ ./skill14
=====
                OSSP - SKILL 14
            APPEND & STDERR REDIRECTION
=====

[PART 1 - APPEND REDIRECTION]

Enter a command using >>
Example: echo SecondLine >> append.txt

OSSP-Append> echo FirstLine >> append.txt

[APPEND PARSING]
Command : echo FirstLine
File    : append.txt

[APPEND FILE]
File opened successfully.
Append mode enabled.
File descriptor: 3

[APPEND VALIDATION]
Command executed successfully.
Existing data was preserved.
New data was appended.

[APPEND FILE CONTENT]
-----
FirstLine
-----

=====
[PART 2 - STDERR REDIRECTION]
=====

Enter a command using 2>
Example: ls /invalid/path 2> errors.txt

OSSP-Stderr> ./skill14

[ERROR]
Invalid stderr syntax.
Use: command 2> filename
hemasai@hemasai:~/OSSP_Assignment$ |

```

15. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

```

hemasai@hemasai:~/OSSP_Assignment$ ./skill15
=====
OSSP - SKILL 15
COMBINED REDIRECTION & COMMANDS
=====

[SUPPORTED FEATURES]
Pipes: |
Input: <
Output: >
Error: 2>
File descriptor duplication: dup2()

Examples:
cat < input.txt | grep Hello > output.txt
ls /wrong 2> errors.txt
cat < input.txt | grep Hello > output.txt 2> errors.txt

OSSP-Skill15> cat < input15.txt | grep Hello > output15.txt

=====
[EXECUTION PLAN]
=====
Total pipeline stages: 1

Stage 1
  Command: cat
  Arguments: cat
  stdin  <- input15.txt

[SYNTAX VALIDATION]
Stage 1 syntax: VALID
Combined command syntax: VALID

=====
[INTEGRATED EXECUTION]
=====

[PIPE CREATION]

[PROCESS CREATION]
Stage 1 process PID: 2794

[DESCRIPTOR CLEANUP]

[PROCESS SYNCHRONIZATION]
Hello OSSP
Stage 1 exit status: 0

=====
SKILL 15 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$ |

```

-
16. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

```
hemasai@Hemasai:~/OSSP_Assignment$ gcc -Wall -Wextra skill16.c -o skill16
hemasai@Hemasai:~/OSSP_Assignment$ ./skill16
```

```
=====
OSSP - SKILL 16
BACKGROUND JOBS
=====
```

```
[JOB TABLE]
```

```
Job table initialized.
Maximum jobs: 20
```

```
[BACKGROUND JOB SUPPORT]
```

```
Use '&' at the end of a command.
Example: sleep 5 &
```

```
OSSP-Skill16> sleep 10 &
```

```
[JOB STORED]
```

```
Job ID : 1
PID    : 2835
State  : RUNNING
Command: sleep 10
```

```
[BACKGROUND PROCESS]
```

```
Process launched without blocking.
Parent prompt returned immediately.
```

```
OSSP-Skill16> jobs
```

```
=====
[JOB MONITOR]
=====
```

```
[JOB TABLE STATUS]
```

```
=====
JOB ID  PID    STATE    COMMAND
-----
1       2835    RUNNING  sleep 10
=====
```

```
OSSP-Skill16> jobs
```

```
=====
[JOB MONITOR]
=====
```

```
[JOB COMPLETED]
```

```
Job ID : 1
PID    : 2835
Exit status: 0
```

```
[JOB TABLE STATUS]
```

```
=====
JOB ID  PID    STATE    COMMAND
-----
1       2835    COMPLETED  sleep 10
=====
```

```
OSSP-Skill16> cleanup
```

```
[JOB REMOVED]
```

```
Job ID 1 removed from job table.
```

JOB ID	PID	STATE	COMMAND
1	2835	COMPLETED	sleep 10

OSSP-Skill16> cleanup
 [JOB REMOVED]
 Job ID 1 removed from job table.

=====

[JOB TABLE STATUS]
 =====

No active jobs.

OSSP-Skill16> sleep 5 &
 [JOB STORED]
 Job ID : 2
 PID : 2836
 State : RUNNING
 Command: sleep 5

[BACKGROUND PROCESS]
 Process launched without blocking.
 Parent prompt returned immediately.

OSSP-Skill16> sleep 8 &
 [JOB STORED]
 Job ID : 3
 PID : 2839
 State : RUNNING
 Command: sleep 8

[BACKGROUND PROCESS]
 Process launched without blocking.
 Parent prompt returned immediately.

[JOB COMPLETED]
 Job ID : 2
 PID : 2836
 Exit status: 0

OSSP-Skill16> sleep 12 &
 [JOB STORED]
 Job ID : 4
 PID : 2840
 State : RUNNING
 Command: sleep 12

[BACKGROUND PROCESS]
 Process launched without blocking.
 Parent prompt returned immediately.

OSSP-Skill16> jobs

=====

[JOB MONITOR]
 =====

=====

[JOB TABLE STATUS]
 =====

JOB ID	PID	STATE	COMMAND
--------	-----	-------	---------

PID : 2836
State : RUNNING
Command: sleep 5

[BACKGROUND PROCESS]
Process launched without blocking.
Parent prompt returned immediately.

OSSP-Skill16> sleep 8 &
[JOB STORED]
Job ID : 3
PID : 2839
State : RUNNING
Command: sleep 8

[BACKGROUND PROCESS]
Process launched without blocking.
Parent prompt returned immediately.

[JOB COMPLETED]
Job ID : 2
PID : 2836
Exit status: 0

OSSP-Skill16> sleep 12 &
[JOB STORED]
Job ID : 4
PID : 2840
State : RUNNING
Command: sleep 12

[BACKGROUND PROCESS]
Process launched without blocking.
Parent prompt returned immediately.

OSSP-Skill16> jobs

=====
[JOB MONITOR]
=====

=====
[JOB TABLE STATUS]
=====

JOB ID	PID	STATE	COMMAND
2	2836	COMPLETED	sleep 5
3	2839	RUNNING	sleep 8
4	2840	RUNNING	sleep 12

OSSP-Skill16> validate

[JOB TABLE VALIDATION]
Job table is valid.
All job records are consistent.

[JOB COMPLETED]
Job ID : 3
PID : 2839
Exit status: 0

OSSP-Skill16> |

OSSP-Skill16> validate
[JOB TABLE VALIDATION]
Job table is valid.
All job records are consistent.

[JOB COMPLETED]
Job ID : 3
PID : 2839
Exit status: 0

OSSP-Skill16> validate

[JOB TABLE VALIDATION]
Job table is valid.
All job records are consistent.

[JOB COMPLETED]
Job ID : 4
PID : 2840
Exit status: 0

OSSP-Skill16> |

```

JOB ID : 4
PID    : 2840
Exit status: 0

OSSP-Skill16> exit

[EXIT]
Final job table:

=====
[JOB TABLE STATUS]
=====
JOB ID  PID    STATE    COMMAND
-----
2       2836    COMPLETED  sleep 5
3       2839    COMPLETED  sleep 8
4       2840    COMPLETED  sleep 12

[JOB TABLE VALIDATION]
Job table is valid.
All job records are consistent.

=====
                SKILL 16 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$

```

17. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

```

hemasai@Hemasai:~/OSSP_Assignment$ warning: unused variable 'input' [-Wunused-
warning: command not found
hemasai@Hemasai:~/OSSP_Assignment$ ./skill17
=====
OSSP - SKILL 17
=====
[JOB TABLE]
Job table initialized successfully.

[BACKGROUND JOB]
Launching a non-blocking process...

[BACKGROUND JOB]
Job ID : 1
PID    : 2920
State  : RUNNING
Prompt returned immediately.

[ACTIVE JOBS]
=====
Job ID  PID    State
-----
1       2920    RUNNING
=====

[JOB STATUS]
Job ID : 1
PID    : 2920
State  : RUNNING

[JOB ID]
Retrieved Job ID = 1

[JOB MONITOR]

[JOB STATUS]
Job ID : 1
PID    : 2920
State  : COMPLETED

[JOB LIST]

[ACTIVE JOBS]
=====
Job ID  PID    State
-----
No active jobs remaining.
=====

[FOREGROUND SWITCH TEST]
=====

[BACKGROUND JOB]
Job ID : 2
PID    : 2921
State  : RUNNING
Prompt returned immediately.

[FOREGROUND SWITCH]
Target Job ID: 2
Target PID: 2921
Transferring terminal control...
Terminal control transferred.
Resuming process...
Waiting for job completion...

[JOB STATE UPDATED]
Job ID : 2
PID    : 2921
State  : COMPLETED

[VALIDATION]
Job records validated successfully.

=====
                SKILL 17 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$

```

18. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

```
nemasai@Hemasai:~/OSSP_Assignment$ ./skill18
```

```
=====
                OSSP - SKILL 18
                SIGNAL & JOB RECOVERY
=====

[JOB TABLE]
Job table initialized.

=====
[SIGNAL INFORMATION]
=====
SIGINT   : 2
SIGTERM  : 15
SIGSTOP  : 19
SIGCONT  : 18
SIGTSTP  : 20

SIGSTOP stops a process.
SIGCONT resumes a stopped process.
SIGTSTP is normally generated by Ctrl+Z.

[SIGNAL MASK]
SIGINT blocked: NO
SIGTERM blocked: NO
SIGCONT blocked: NO
SIGTSTP blocked: NO

=====
[SIGNAL BEHAVIOR TEST]
=====
[PARENT PROCESS]
Parent PID: 2959
Child PID: 2960

[PROCESS GROUP]
Process Group ID: 2960

Stopping child using SIGSTOP...

[JOB STATUS UPDATED]
Job ID : 1
PID    : 2960
```

```

[JOB STATUS UPDATED]
Job ID : 1
PID    : 2960
State  : STOPPED

=====
[JOB TABLE STATUS]
=====
JOB ID  PID    STATE    PGID
-----
1       2960   STOPPED   2960

=====
[BACKGROUND RECOVERY]
=====
Target Job ID: 1
Target PID: 2960
Process Group: 2960

[CONTINUE SIGNAL]
SIGCONT sent successfully.

[JOB STATE]
Job ID : 1
State  : RUNNING

Waiting for process completion...
[CHILD PROCESS]
Child PID: 2960
Process Group ID: 2960
Child is running.
Child working... 1/5
Child working... 2/5
Child working... 3/5
Child working... 4/5
Child working... 5/5
Child completed normally.

[JOB COMPLETED]
Exit status: 0

=====
[JOB TABLE STATUS]
=====
JOB ID  PID    STATE    PGID
-----
1       2960   COMPLETED 2960

[VALIDATION]
Job table is valid.
All signal and job records are consistent.

=====
                SKILL 18 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$ |

```

-
19. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

```

hemasai@Hemasai:~/OSSP_Assignment$ ./skill19
=====
OSSP - SKILL 19
SIGNAL INTERRUPT HANDLING
=====

[SIGNAL HANDLER]
SIGINT handler registered successfully.
Shell will remain active after Ctrl+C.
[SHELL PROTECTION]
SIGTSTP protection configured.

=====

[SIGNAL INFORMATION]
=====
SIGINT = 2
SIGTERM = 15
SIGTSTP = 20
SIGCONT = 18

SIGINT is normally generated by Ctrl+C.
The shell catches SIGINT and forwards it.
The foreground child uses default SIGINT behavior.

[ SIGNAL ACTION VERIFICATION ]
SIGINT handler: REGISTERED
SIGTSTP shell protection: ENABLED

=====

[FOREGROUND JOB]
=====
Launching foreground job...
Command: sleep 30

[PARENT / SHELL]
Shell PID: 3003
Foreground PID: 3004
Foreground process group: 3004
Shell is protected from SIGINT.
Waiting for foreground job...
[CHILD PROCESS]
Child PID: 3004
Process Group ID: 3004
Child is running.
Press Ctrl+C to send SIGINT.

^C

[SIGINT RECEIVED]
Shell received SIGINT.
Forwarding SIGINT to foreground job...

[JOB TERMINATED]
Signal: 2
Foreground job terminated by SIGINT.

[JOB STATE UPDATED]
State: TERMINATED BY INTERRUPT

=====

[FINAL SIGNAL TEST]
=====
SIGINT was successfully captured.
Signal was forwarded to foreground job.
Shell remained active.
Foreground job was terminated.

=====
SKILL 19 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$ |

```

20. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

```
hacker@hacker:~/OSDP_Assignments$ gcc -Wall -Werror shell.c -o shell
hacker@hacker:~/OSDP_Assignments$ ./shell128

=====
OSDP - SHELL 28
SIGTSTP & JOB SUSPENSION
=====

[SHELL INITIALIZATION]
Shell PID : 3843
Shell PGID: 3843

[SIGNAL CONFIGURATION]
SIGCHLD handler registered.
SIGTSTP protected in shell.
Terminal job-control signals configured.

[FOREGROUND JOB]
Starting test process...
The process will run for 15 seconds.
Press Ctrl+Z to suspend it.

[TERMINAL CONTROL]
Terminal transferred to PGID: 3843
Waiting for job state change...

[CHILD PROCESS]
Child PID : 3843
Child PGID: 3843
Child has terminal control.

Child working... 1/15
Child working... 2/15
Child working... 3/15
Child working... 4/15
Child working... 5/15
Child working... 6/15
Child working... 7/15
Child working... 8/15
Child working... 9/15
Child working... 10/15
Child working... 11/15
Child working... 12/15
Child working... 13/15
^Z

[SIGTSTP CAPTURED]
Foreground process was suspended.
Stopping signal: 28

[JOB STATE UPDATED]
Job ID : 1
PID   : 3843
PGID  : 3843
State : STOPPED

[TERMINAL CONTROL RESTORED]
Terminal returned to shell.

[JOB TABLE]
-----
Job ID : 1
PID   : 3843
PGID  : 3843
State : STOPPED
-----

Press ENTER to resume the stopped job...

[RESUME STOPPED JOB]
-----
Job ID : 1
PID   : 3843
PGID  : 3843
-----

Sending SIGCONT.
SIGCONT sent successfully.

[JOB STATE UPDATED]
State: RUNNING
[TERMINAL CONTROL]
Terminal transferred back to PGID: 3843
Waiting for completion...
Child working... 14/15
Child working... 15/15
^Z

[TERMINAL CONTROL RESTORED]
Shell owns the terminal again.

[JOB TABLE]
-----
Job ID : 1
PID   : 3843
PGID  : 3843
State : RUNNING
-----

[VALIDATION]
-----
Validation failed
-----

=====
SHELL 28 EXECUTION COMPLETED
=====
hacker@hacker:~/OSDP_Assignments$
```

21. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```

=====
SKILL 20 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$ nano skill21.c
hemasai@Hemasai:~/OSSP_Assignment$ gcc -Wall -Wextra skill21.c -o skill21
hemasai@Hemasai:~/OSSP_Assignment$ ./skill21
=====
OSSP - SKILL 21
MODULE INTEGRATION
=====

[MODULE INITIALIZATION]
Parent module initialized.
Child module prepared.

[INTERFACE VALIDATION]
Parent-child interface verified.
Process communication interface is valid.

[CHILD MODULE]
Parent waiting for child module...
Child module started successfully.
Child PID: 3082
Child module execution completed.
Child PID: 3082
Child module completed successfully.
Child exit status: 0

[END-TO-END TEST]
Child module completed successfully.
End-to-end execution verified.

[RELIABILITY CHECK]
All connected modules completed normally.
System execution is reliable.

[REFACTOR CHECK]
Child functionality maintained as a separate module.
Module interface remains valid.

[ERROR HANDLING TEST]
Invalid module detected.
Error handled gracefully.
Invalid input detected.
Execution recovered without terminating.

=====
[FINAL STATUS]
=====
Interface validation : PASSED
End-to-end test      : PASSED
Reliability check    : PASSED
Refactor check       : PASSED

=====
SKILL 21 EXECUTION COMPLETED
=====

```

-
22. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

```

hemasai@Hemasai:~/OSSP_Assignment$ ./skill22
=====
                OSSP - SKILL 22
                VALGRIND DEMO
=====

[MEMORY ALLOCATION]
Memory allocated successfully.
Stored value: 1 10 Item_1
Stored value: 2 20 Item_2
Stored value: 3 30 Item_3
Stored value: 4 40 Item_4
Stored value: 5 50 Item_5

[TEST VALIDATION]
All allocated data verified successfully.

[MEMORY RELEASE]
Allocated memory released successfully.

[CLEANUP VERIFICATION]
Memory pointers cleared.
Cleanup verified successfully.

[ERROR SCENARIO TEST]
Temporary test allocation successful.
Temporary test memory released safely.

[REPORT]
Test report generated: skill22_report.txt

[RESULT]
No intentional memory leak remains.
Memory management completed successfully.
=====
                SKILL 22 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$ |

```

-
23. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

```
hemasai@Hemasai:~/OSSP_Assignment$ ./skill23
=====
OSSP - SKILL 23
PIPELINE STRESS DEMO
=====

[PIPELINE]
Launching multiple pipeline jobs...
Number of jobs: 4

[CHILD JOB 2]
PID : 3158
Status : Running
[CHILD JOB 3]
PID : 3159
Status : Running
[CHILD JOB 1]
PID : 3157

[PARENT PROCESS]
Status : Running
All pipeline jobs launched.
Monitoring child processes...

[CHILD JOB 4]
PID : 3160
Status : Running
Child PID 3160 : SUCCESS
Child PID 3158 : SUCCESS
Child PID 3159 : SUCCESS
Child PID 3157 : SUCCESS
Job 1 completed successfully.
Job 2 completed successfully.
Job 3 completed successfully.
Job 4 completed successfully.

[STABILITY CHECK]
All pipeline jobs were monitored.
All child processes completed.

[RESOURCE MONITOR]
Parent PID : 3156
Jobs launched : 4
Jobs completed : 4
User CPU time : 0.001564 seconds
System CPU time : 0.003859 seconds
Maximum RSS : 1088 KB
Page faults : 0
Context switches : 4

[PERFORMANCE]
Execution time : 0.031 seconds
Pipeline throughput: successful

[FAILURE CHECK]
No child process failure detected.

[JOB SUMMARY]
=====
JOB ID PID STATE EXIT
=====
1 3157 SUCCESS 0
2 3158 SUCCESS 0
3 3159 SUCCESS 0
4 3160 SUCCESS 0
=====

[ANALYSIS]
Multiple jobs executed successfully.
Pipeline workload completed normally.
Resource monitoring completed.
No execution failure detected.
=====
SKILL 23 EXECUTION COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$ |
```

24. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines,
Demonstrate Signals, Final Project Review.

```
hemasai@Hemasai:~/OSSP_Assignment$ ./skill24
```

```
=====
OSSP - SKILL 24
PROJECT REVIEW DEMO
=====
```

```
[SOURCE CODE]
Source code organization verified.
```

```
[BUILD SCRIPT]
Build configuration verified.
GCC build configuration available.
```

```
[README]
README not found.
Creating project README...
Project README created successfully.
```

```
[REPOSITORY]
Repository structure verified.
```

```
[DELIVERABLES]
Skill source files found: 23/24
Project files validated successfully.
```

```
[PIPELINE DEMO]
Pipeline demonstration prepared.
Multiple process execution reviewed.
Pipeline workflow validated.
```

```
[SIGNAL DEMO]
Signal handling demonstration reviewed.
SIGINT and SIGTSTP workflows validated.
Job control behavior reviewed.
```

```
[RELEASE]
Release structure prepared successfully.
```

```
[FINAL PROJECT REVIEW]
All project components validated.
Final project review passed.
```

```
=====
SKILL 24 DEMO COMPLETED
=====
hemasai@Hemasai:~/OSSP_Assignment$ |
```